CSE 221: ALGORITHMS

COMPLETE EXAM SOLUTIONS

BRAC University
Department of Computer Science and Engineering
Mid-Semester & Other Exams

Generated: September 11, 2026

SPRING 2024 - SET A

Question 1a (3 marks marks)

Asymptotic Upper Bounds & Growth Rate Ordering
Functions: f■(n)=log(n!), f■(n)=n²log_n(n^n), f■(n)=n■+10n², f■(n)=17.13^n, f■(n)=nlogn, f■(n)=n·■√(n²)

Function	Bound	Explanation
f■(n)=log(n!)	O(n log n)	Stirling: log(n!)≈nlogn
f■(n)=n²log■(n^n)	O(n²)	log■(n^n)=n, so n²·n/n=n²
f■(n)=n■+10n²	O(n■)	Dominant term
f■(n)=17.13^n	O(17.13^n)	Exponential
f■(n)=n log n	O(n log n)	Direct
f■(n)=n·■√(n²)	O(n^1.4)	n·n^0.4=n^1.4

Sorted Order (ascending): f■(nlogn) < f■(n^1.4) < f■(nlogn) < f■(n²) < f■(n■) < f■(17.13^n)

Question 1b (3 marks marks)

Code: for i in range(1,n): j=1; while j<i²: j=j-1
Issue: j decreases infinitely. Assuming j=j+1 instead.

Inner loop runs i² times for each i. Total: ∑(i²) for i=1 to n-1 = (n-1)n(2n-1)/6 = O(n³)
Answer: O(n³)

Question 1c (4 marks marks)

Option 1: T(n) = 8T(n/4) + n√n

a=8, b=4, f(n)=n^1.5
log■(8)=log■(2³)=3/2=1.5
f(n)=Θ(n^1.5)=Θ(n^log■8) → Case 2 of Master Theorem
T(n) = Θ(n^1.5 · log n)

Option 2: T(n) = 2T(n-2) + 1

Linear recurrence (not standard form). Expanding:
T(n) = 2T(n-2)+1 = 2[2T(n-4)+1]+1 = 4T(n-4)+3
After k=(n-1)/2 steps: T(n) = 2^k·T(1) + (2^k-1)
T(n) = Θ(2^(n/2))

Question 2 (5 marks marks)

Karatsuba's Fast Multiplication Algorithm

Part i - Mathematical Proof:
For N-digit numbers x and y:
x = x■·10^(N/2) + x■
y = y■·10^(N/2) + y■
x·y = x■y■·10^N + (x■y■ + x■y■)·10^(N/2) + x■y■

Key insight: (x■y■ + x■y■) = (x■+x■)(y■+y■) - x■y■ - x■y■
Instead of 4 multiplications, we compute only 3:
1. x■·y■
2. x■·y■
3. (x■+x■)(y■+y■)
Recurrence: T(n) = 3T(n/2) + O(n)
Solution: T(n) = O(n^log■3) = O(n^1.585)
Speedup over O(n²) is significant for large numbers.

Part ii - Binary Numbers:
Yes, the algorithm works for binary numbers.
Instead of base 10^(N/2), use base 2^(N/2). All mathematical principles remain identical.
The base change doesn't affect the core logic, only the digit positions.

Question 3a (4 marks marks)

Quick Sort Worst Case: O(n²)
Example (last element pivot): [1, 2, 3, 4, 5, 6, 7, 8]

Pivot=8 (largest). All elements go left. Partition: (7,0)
$T(n) = T(n-1) + T(0) + O(n) = T(n-1) + O(n) = O(n^2)$

Question 3b (6 marks marks)

Finding Peak Index in Bitonic Array
Array: increasing [0,i) then increasing [i,N) where [0,i) values > [i,N) values
Example: [9, 12, 15, 2, 4, 5, 7, 8] with i=3

Algorithm:
Binary search comparing arr[mid] with arr[mid+1]
If arr[mid] > arr[mid+1]: peak is in left half
Else: peak is in right half

```
Algorithm FindIndex(arr, n): left = 0, right = n-1 while left < right: mid = (left + right) / 2 if arr[mid] > arr[mid+1]: right = mid else: left = mid + 1 return left Time: O(log n), Space: O(1)
```

Question 4 (10 marks marks)

File Navigator System using Graph

i. Adjacency List:

```
Drive: {Work, Personal, otherFile} Work: {Tasklist, Schedule} Tasklist: {task1.txt} Personal: {Songs} Schedule: {ToDolist} otherFile: {Media} Media: {Cinema} Cinema: {HorrorMovies, ActionMovie} Songs: {Cinema}
```

ii. Preferred Algorithm: DFS (Depth-First Search)
Reason: Finds complete path from root to target. Ideal for hierarchical navigation.

iii. DFS Simulation for 'Horror Movies':

```
Step 1: Start at Drive, explore children [Work, Personal, otherFile] Step 2: Visit Work → Tasklist → task1.txt (dead end, backtrack) Step 3: Visit Work → Schedule → ToDolist (dead end, backtrack) Step 4: Backtrack to Drive, visit Personal → Songs → Cinema Step 5: At Cinema, find neighbors [HorrorMovies, ActionMovie] Step 6: Found HorrorMovies! Path: Drive → Personal → Songs → Cinema → Horror Movies Time: O(V+E), Space: O(V) for recursion
```

SPRING 2024 - SET B

Question 1a (3 marks marks)

Modified Functions:
f■ changed to: n·■√(n²) = n^1.5 → O(n^1.5)
f■ changed to: n³ + 10n² → O(n³)
f■ changed to: n²log■(n^n) → O(n²)
New sorted order: f■ < f■ < f■ < f■ < f■ < f■

Question 1b (3 marks marks)

Code now: for i in range(1,n): while j<i²: j=j+1
Answer: O(n³) (same analysis as Set A)

Question 1c (4 marks marks)

T(n)=4T(n/4)+√n: log■(4)=1, f(n)=n^0.5 → **O(n log n)**
T(n)=T(n-3)+n: Expanding k=n/3 times → **O(n²)**

Question 2 (5 marks marks)

Part ii: Hexadecimal Multiplication
Yes, Karatsuba works for hexadecimal numbers.
Split by 16^(N/2) instead of 10^(N/2). The algorithm is base-independent.

Question 3a (4 marks marks)

Quick Sort with first element pivot:
Example: [8, 7, 6, 5, 4, 3, 2, 1]
Pivot=8 (largest). All elements go left → O(n²)

Question 3b (6 marks marks)

Decreasing then Increasing Array:
Array: [5, 4, 1, 12, 10, 9, 7, 6] with i=3
Find transition point. Same binary search algorithm applies.
Time: O(log n), Space: O(1)

Question 4 (10 marks marks)

Different graph structure:
i. Adjacency matrix: 8×8 with 1s for edges
ii. Bipartite: Use 2-coloring with BFS. Answer depends on graph structure.
iii. Simulation: Show complete BFS traversal with queue and visited set

SUMMER 2024 - SET A

Question 1a (5 marks marks)

Recursive Function Analysis:
func: 3 calls to F(n/2), plus func2(n)
func2: Nested loops. Outer: n to 1 by 4 → n/4 iterations
Inner: 0 to n by 2 → n/2 iterations
func2 time: O(n/4 × n/2) = O(n²)

Recurrence: T(n) = 3T(n/2) + O(n²)
log■(3) ≈ 1.585 < 2
f(n) = n² = Ω(n^(1.585+ε))
By Master Theorem Case 3: **T(n) = Θ(n²)**

Question 1b (3 marks marks)

3n² vs n·■√n = n^1.25
n² > n^1.25 → **3n² = Ω(n^1.25)**
30√n vs n²: √n < n² → **30√n = O(n²)**

Question 1c (2 marks marks)

T(n) = T(√n) + O(n) - Can we use Master Theorem?
Answer: No
Master Theorem requires T(n) = aT(n/b) + f(n) form.
T(√n) doesn't fit this pattern (non-integer reduction).
Solve via substitution: m = log■n, get T(2^m) = T(2^(m/2)) + 2^m

Question 2 (7 marks marks)

Square Root Finding Problem

Algorithm P: Binary Search (O(√n))

```
Algorithm FindSqrtBinary(key): if key == 0: return 0 left = 1, right = key, result = 1 while left <= right: mid = (left + right) / 2 if mid * mid == key: return mid else if mid * mid < key: result = mid left = mid + 1 else: right = mid - 1 return result Complexity: O(log key)
```

Algorithm R: Optimized Binary Search (O(log n))

```
Algorithm FindSqrtOptimized(key): if key < 2: return key left = 1 right = key // 2 // √key ≤ key/2 for key > 2 while left <= right: mid = (left + right) / 2 square = mid * mid if square == key: return mid else if square < key: left = mid + 1 else: right = mid - 1 return right Complexity: O(log key) with tighter bounds
```

Question 3a (5 marks marks)

Maximum Subarray: [-3, 7, 12, -8, -2, 83, -7, 4]
Using Kadane's Algorithm:

Index	-3	7	12	-8	-2	83	-7	4
curr_sum	-3	7	19	11	9	92	85	89
max_sum	-3	7	19	19	19	92	92	92

Maximum subarray sum: 92
Best segment: Index 5 (value 83) or indices 1-5 [7,12,-8,-2,83] = 92

3a.ii: Time Complexity = O(n)
Kadane's algorithm passes through array once with constant work per element.

Question 4 (10 marks marks)

Bangladesh Highway Graph Problem

i. Graph Representation (Adjacency List):
Edges: 1-2, 1-5, 2-3, 2-5, 3-4, 3-5, 4-8, 5-6, 5-7, 6-7, 7-8
Space: O(V+E) = O(8+11) = O(19)

ii. Additional Highways:
Complete graph: n(n-1)/2 = 8×7/2 = 28 edges
Current: 8 edges
Need: 28 - 8 = 20 additional highways

iii. Bipartite Check (2-coloring):
Use BFS with colors 0 and 1. If at any point neighbor has same color → NOT bipartite.
This graph contains odd cycles (e.g., 2-3-5-2) → **NOT BIPARTITE**

iv. Odd Cycle Detection:
Graph has odd cycles iff NOT bipartite.
2-coloring algorithm will fail. Found triangle: nodes 2, 3, 5.

SUMMER 2024 - SET B

Question 1a (5 marks marks)

Modified: F calls 4 times F(n/4) instead of 3 times F(n/2)
func2: for(int k=n; k>=1; k-=3) - changes iteration count
Analysis: func2 = O((n/3)×(n/2)) = O(n²)
Recurrence: T(n) = 4T(n/4) + O(n²)
log■(4) = 1, f(n) = n² > n¹
T(n) = Θ(n²)

Question 1b (3 marks marks)

3n² vs n·■√n = n^1.25 → **3n² = Ω(n^1.25)**
30√n vs n²: √n < n² → **30√n = O(n²)**

Question 3a (8 marks marks)

Array: [2, -6, 8, 8, -9, -1, -2, 20]
Find best consecutive segment for positive impact.
Same Kadane's algorithm: O(n) time
Best segment: [8, 8] or [8, 8, -9, -1, -2, 20] = 24

Question 4 (10 marks marks)

9 highways instead of 8
Additional highway changes connectivity analysis but same methodology.
i. Adjacency list with 9 edges
ii. Completeness analysis with 9 edges
iii. Bipartite check and odd cycle detection remain same type of analysis

FALL 2024 - SET A

Question 1a (4 marks marks)

Code Analysis:
F makes 3 calls to F(n/2)
A(n) is linear: O(n)
Recurrence: T(n) = 3T(n/2) + O(n)
log■(3) ≈ 1.585, n < n^1.585
By Master Theorem Case 1: **T(n) = Θ(n^log■3) = Θ(n^1.585)**

Question 1b (5 marks marks)

Peak Element Finding in Array
Element > adjacent elements
Validation:
[1,5,4,3,2]→True (5 is peak)
[1,3,2,5,4]→True (3 and 5 are peaks)
[5,2,1,3,4]→True (5 is peak)
[1,2,3,4,5]→False (monotonic increasing)

O(log n) Algorithm using Binary Search:

```
Algorithm FindPeak(arr, n): left = 0, right = n-1 while left < right: mid = (left + right) / 2 if arr[mid] > arr[mid+1]: right = mid else: left = mid + 1 return left Works because: If mid > mid+1, peak is ≤ mid If mid ≤ mid+1, peak is > mid+1
```

Multiple peaks: Algorithm finds one peak. For all peaks, need O(n) scan.

Question 2a (5 marks marks)

Warehouse Sorting Problem - Stable Sort Requirement
Recommended Algorithm: Merge Sort
Why: O(n log n) guaranteed, stable, predictable performance

```
Algorithm MergeSort(arr, left, right): if left < right: mid = (left + right) / 2 MergeSort(arr, left, mid) MergeSort(arr, mid+1, right) Merge(arr, left, mid, right) Algorithm Merge(arr, left, mid, right): Create temp arrays and merge while maintaining stability: - Compare elements from left and right halves - Take smaller element, ensuring equal-deadline packages maintain original order (stability) Complexity: O(n log n), Space: O(n) Stability: Guaranteed - equal priorities preserve order
```

Question 2b (5 marks marks)

Memory-Constrained Solution
i. Suggested Algorithm: Randomized Quick Sort
O(1) extra space, expected O(n log n)
Worst case O(n²) occurs with worst-case pivot selection (very rare with randomization)

ii. Guarantee O(n log n) in all cases:
Solution: Randomized Quicksort with Median-of-Medians
Modified approach:
• Use Median-of-Medians for guaranteed good pivot
• Ensures balanced partitions (n/3 - 2n/3)
• Guarantees O(n log n) worst case
• Same O(1) space as quicksort
• More complex but deterministically O(n log n)

Question 3 (10 marks marks)

Maximum Subarray Problem using Divide & Conquer
Array: [1, 4, 3, -5, 5, 6, 1, -4]

Part i: Cross-Sum Analysis
Cross-sum = sum crossing the midpoint
Count when: Cross-Sum > LeftMax AND Cross-Sum > RightMax
Excluding base cases (size 1 subarrays)

```
Split at mid (3,4): Left [1,4,3,-5]: LeftMax=8 Right [5,6,1,-4]: RightMax=12 CrossSum: (-5)+5+6=6 < both Split at mid (1,2): Left [1]: 1 Right [4,3,-5,5,6,1,-4]: 14 CrossSum: 1+4+3+(-5)+5+6+1=15 > 14? No Count: Typically 1-2 times through recursion
```

Part ii: Divide & Conquer Algorithm O(n log n):

```
Algorithm MaxSubarrayDC(arr, left, right): if left == right: return (arr[left], left, right) mid = (left + right) / 2 (maxL, l1, r1) = MaxSubarrayDC(arr, left, mid) (maxR, l2, r2) = MaxSubarrayDC(arr, mid+1, right) (maxC, l3, r3) = MaxCrossing(arr, left, mid, right) if maxL ≥ maxR and maxL ≥ maxC: return (maxL, l1, r1) else
```

```
if maxR ≥ maxL and maxR ≥ maxC: return (maxR, l2, r2) else: return (maxC, l3, r3) Algorithm MaxCrossing(arr, left, mid, right): leftSum = 0, maxLeft = NEG_INF for i = mid down to left: leftSum += arr[i] if leftSum > maxLeft: maxLeft = leftSum midIdx = i rightSum = 0, maxRight = NEG_INF for i = mid+1 to right: rightSum += arr[i] if rightSum > maxRight: maxRight = rightSum maxRIdx = i return (maxLeft + maxRight, midIdx, maxRIdx) Time: O(n log n) - T(n) = 2T(n/2) + O(n)
```

FALL 2024 - SET B

Question 1a (4 marks marks)

Modified: F calls F(n/3) FOUR times
A(n) computes sum 1 to n = n(n+1)/2 = O(n)
Recurrence: T(n) = 4T(n/3) + O(n)
log■(4) ≈ 1.262
Since n < n^1.262: **T(n) = Θ(n^log■4) = Θ(n^1.262)**

Question 1b (5 marks marks)

Valley Element (minimum not maximum)
Element < adjacent elements
[5,3,2,1,4]→True (1 is valley)
[3,1,5,2,4]→True (1 is valley)
[1,2,3,4,5]→False (monotonic)
[1,3,5,4,2]→False (no valley)
Algorithm: Same binary search, but check arr[mid] < arr[mid+1]

Question 3 (10 marks marks)

Stock Trading Problem
Array: [2, 3, 5, -7, 3, 2, 1, -6]
Find consecutive days with maximum positive price increase.
Uses same Divide & Conquer approach as Set A

KEY CONCEPTS & FORMULAS

1. Master Theorem: T(n) = aT(n/b) + f(n)
Let c = log_b(a)
Case 1: f(n) = O(n^(c-ε)) ■ T(n) = Θ(n^c)
Case 2: f(n) = Θ(n^c) ■ T(n) = Θ(n^c log n)
Case 3: f(n) = Ω(n^(c+ε)) ■ T(n) = Θ(f(n))

2. Sorting Algorithm Comparison:

Algorithm	Time	Space	Stable	Notes
Merge Sort	O(nlogn)	O(n)	Yes	Divide & Conquer
Quick Sort	$O(n^2)$ worst	O(1)	No	In-place
Heap Sort	O(nlogn)	O(1)	No	In-place
Insertion	$O(n^2)$	O(1)	Yes	Simple

3. Graph Algorithms:
BFS: O(V+E), finds shortest path, uses queue
DFS: O(V+E), finds path deeply, uses stack/recursion
Bipartite: 2-coloring with BFS/DFS
Cycle detection: DFS with colors (white/grey/black)

4. Important Time Complexities:
Linear: O(n)
Logarithmic: O(log n) - binary search
Linearithmic: O(n log n) - good sorts
Quadratic: $O(n^2)$ - nested loops
Cubic: $O(n^3)$ - triple nested loops
Exponential: O(2^n) - subsets
Factorial: O(n!) - permutations

5. Kadane's Algorithm (Maximum Subarray):
Greedy approach, O(n) time, O(1) space
curr_sum = max(arr[i], curr_sum + arr[i])
max_sum = max(max_sum, curr_sum)
Best subarray problem solved optimally

EXAM TIPS & IMPORTANT NOTES

✓ **When to use Merge Sort:**
• Stability required
• Guaranteed O(n log n) needed
• Auxiliary space available
• External sorting

✓ **When to use Quick Sort:**
• In-place sorting critical
• Average case performance sufficient
• Memory very limited
• Cache performance matters

✓ **Binary Search Conditions:**
• Array must be sorted (or specific structure)
• Monotonic properties required
• Can find peak/valley in log n if conditions met
• Pivot-based optimization (like √key for square root)

✓ **Recurrence Relation Solving:**
1. Substitution method: Guess and prove by induction
2. Recursion tree: Visualize work at each level
3. Master Theorem: When T(n) = aT(n/b) + f(n)
Choose method based on recurrence structure

✓ **Common Pitfalls:**
• Confusing O(n) vs Ω(n) vs Θ(n)
• Master Theorem not applicable to all recurrences
• Stability matters more than often realized
• Off-by-one errors in indexing
• Not considering space complexity
• Assuming sorted input when not guaranteed

✓ **Algorithm Selection Checklist:**
■ Time complexity requirement?
■ Space complexity constraint?
■ Stability needed?
■ Input characteristics (sorted, random, adversarial)?
■ Is trade-off acceptable (speed vs simplicity)?
■ Worst-case or average-case critical?

Generated: September 11, 2026 at 18:34